

Introduction to Reinforcement Learning

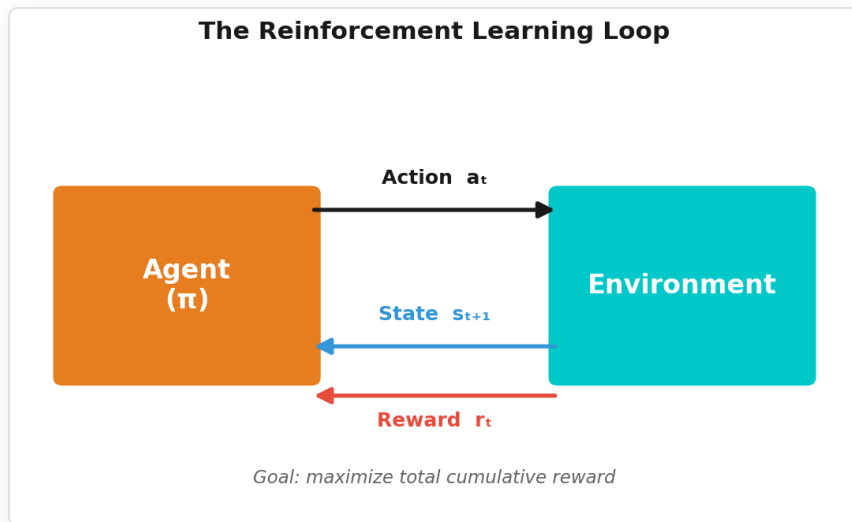
Applied Machine Learning — Session 4, Chapter 1

The Third Paradigm

Paradigm	What the data gives you	Example
Supervised	inputs and the right answer (X, y)	predict house price
Unsupervised	inputs only (X)	find customer groups
Reinforcement	no data set at all — only rewards for actions you try	learn to play a game

RL: an *agent* acts, the *environment* answers with a new situation and a reward. The agent learns from that feedback loop.

The RL Loop



Goal: maximise the **total** reward over an episode — not the next reward.

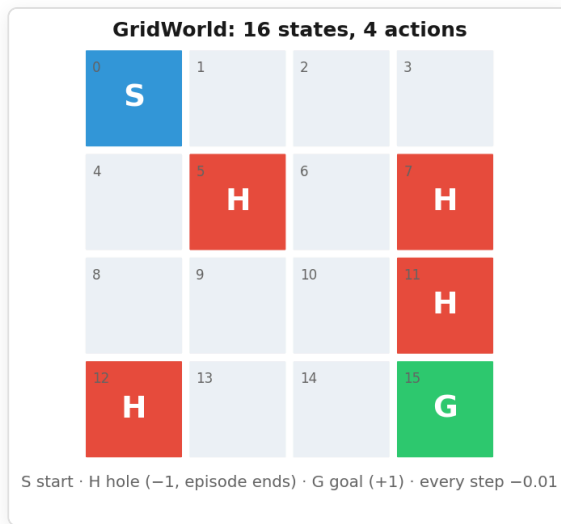
Everyday Analogies

	State	Action	Reward
Training a dog	command + situation	sit / bark / run	treat (or nothing)
Learning to ride a bike	lean angle, speed	steer, pedal	stay up (+) / fall (-)
Video game	screen	controller input	score
Chess	board position	legal move	win / lose at the end

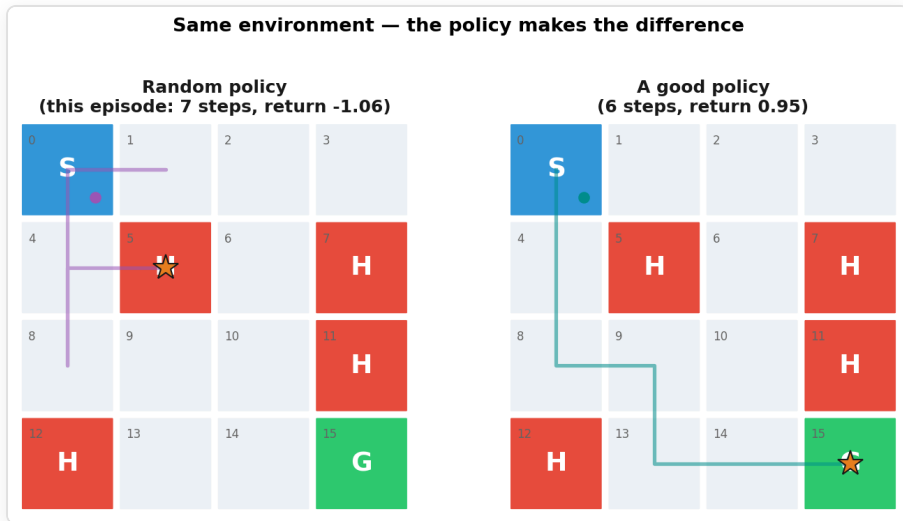
Ask yourself: who tells the chess agent which of its 40 moves was the mistake? → Nobody. That is the **credit assignment problem** — the core difficulty of RL.

Core Vocabulary — on a GridWorld

Term	Meaning	GridWorld
State s	the situation	cell id 0–15
Action a	what the agent can do	$\leftarrow \downarrow \rightarrow \uparrow$
Reward r	feedback	+1 goal, -1 hole, -0.01 per step
Episode	one run	start $\rightarrow$ hole or goal
Return	sum of rewards in an episode	e.g. 6 steps to G: 0.95
Policy π	strategy: state $\rightarrow$ action	a table with 16 entries



Same World, Different Policy



The learning problem: find a policy that maximises the expected return — *without* being told the map.

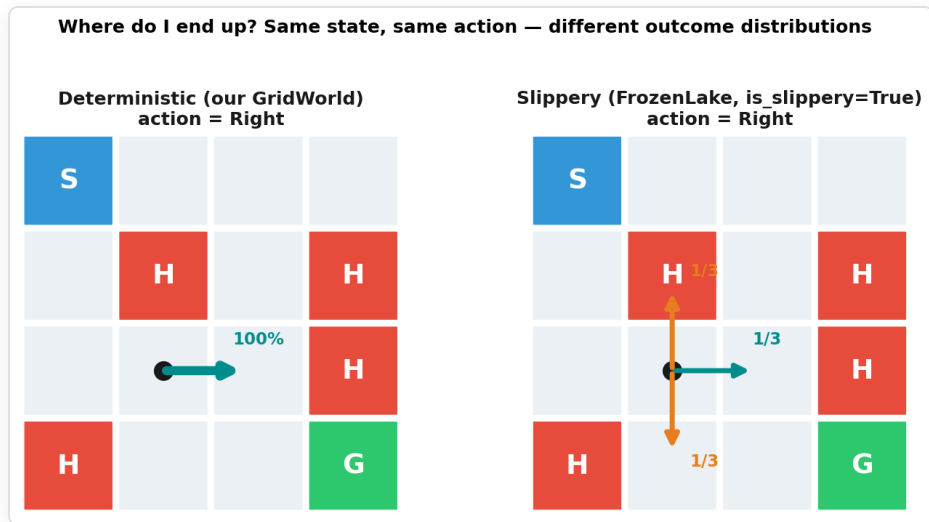
Markov Decision Process (MDP)

The formal frame: (S, A, P, R, γ)

- S states, A actions
- $P(s' | s, a)$ transition probabilities: where do I land?
- $R(s, a)$ reward function
- γ discount factor (0 ... 1)

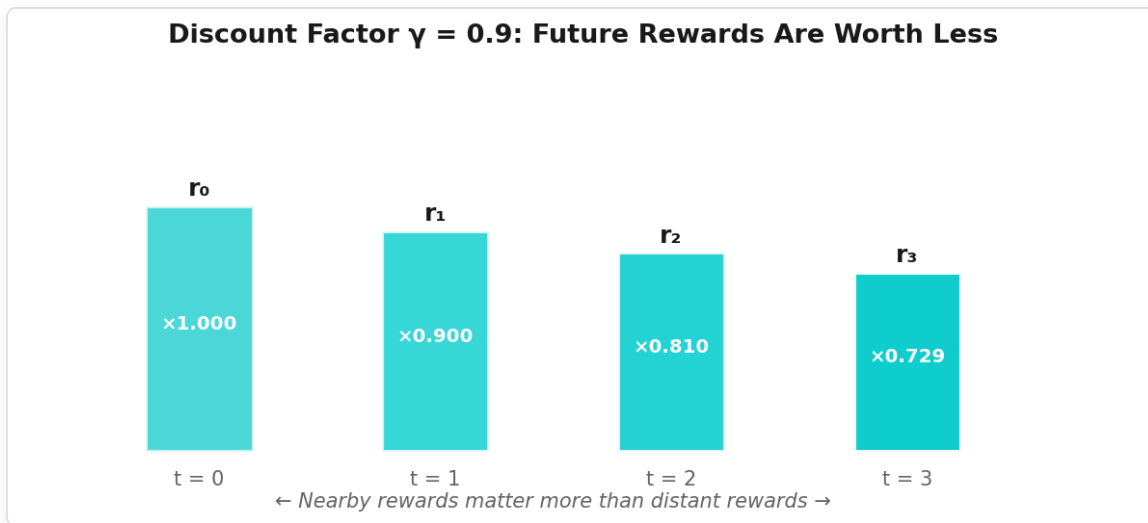
Markov property: *the future depends only on the current state, not on how you got there.* → Like GPS: to plan the route, it needs to know where you are — not where you were an hour ago.

Deterministic vs Stochastic Environments



- Our GridWorld is **deterministic**: `Right` always moves right.
- FrozenLake (Ch11, `is_slippery=True`): intended direction with prob. $\frac{1}{3}$, else it slips sideways.
- $P(s' | s, a)$ captures exactly this. The learning algorithm stays the same — only the numbers get harder.

The Discount Factor γ



- $\gamma = 0$: only the immediate reward counts (short-sighted)
- $\gamma \rightarrow 1$: far-away rewards count almost fully (far-sighted)
- $\gamma = 0.9 \dots 0.99$: typical — we use **0.99** in Ch11

Intuition: €100 today > €100 next year. Same in RL — but γ also lets value *flow backwards* from the goal to earlier

Exploration vs Exploitation

The dilemma every learner faces:

- **Exploit:** do what you already know works best
- **Explore:** try something new — it might be better (or worse)

Restaurant analogy:

exploit → your favourite restaurant (safe, but you never discover a better one)

explore → the new place on the corner (risky, but maybe amazing)

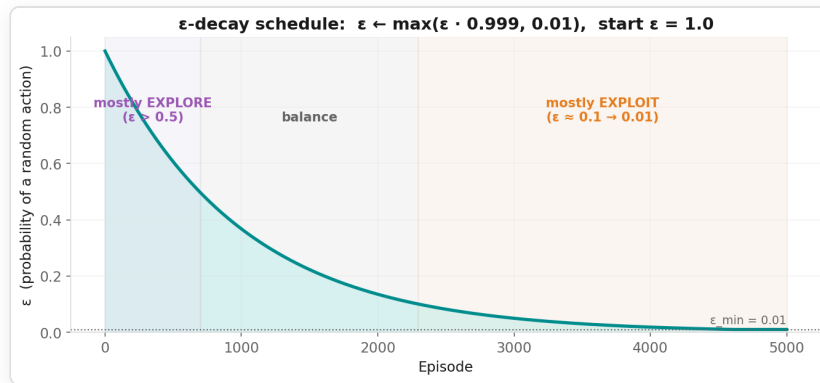
If you only exploit you may be stuck with a mediocre policy forever. If you only explore you never *use* what you learned.

ϵ -Greedy and ϵ -Decay

```
if rng.random() < epsilon:
    action = random_action()      # explore
else:
    action = argmax(Q[state])     # exploit

# after every episode:
epsilon = max(epsilon * 0.999, 0.01)
```

- start $\epsilon = 1$ (know nothing $\rightarrow$ explore everything)
- decay towards $\epsilon = 0.01$ (know the world $\rightarrow$ exploit it)
- too fast: stops exploring before it has learned
- too slow: wastes episodes on random walks



Value: how good is an action in a state?

$Q(s, a)$ = expected total (discounted) reward if I take action a in state s and act well afterwards.

Q-table (16 states $\times$ 4 actions)

	←	↓	→	↑
state 0:	0.90	0.92	0.88	0.90
state 1:	0.90	0.10	0.83	0.88
...				
state 14:	0.96	0.98	1.00	0.95

← next to the goal: → is worth ≈ 1

If you know Q , the policy is trivial: $\pi(s) = \operatorname{argmax}_a Q(s, a)$

→ Chapter 11: learn this table from experience (Q-Learning).

Quick Check

1. A robot vacuum gets +1 for every square metre cleaned and -5 for falling down the stairs. State? Action? Reward? Episode?
2. $\epsilon = 0$ during the whole training. What goes wrong?
3. $\gamma = 0$. Which action does the GridWorld agent prefer in state 14 (next to the goal) — and in state 0?

Quick Check

1. A robot vacuum gets +1 for every square metre cleaned and -5 for falling down the stairs. State? Action? Reward? Episode?
2. $\epsilon = 0$ during the whole training. What goes wrong?
3. $\gamma = 0$. Which action does the GridWorld agent prefer in state 14 (next to the goal) — and in state 0?
 - 1. state = position + dirt map (+ battery), action = move direction / suck, reward as given, episode = one cleaning run until docking.
 - 2. it never explores: it repeats the first action that ever got a non-zero Q (or action 0 forever) → stuck with a bad policy.
 - 3. state 14: → (immediate +1). State 0: every action gives -0.01 now → indifferent, it cannot "see" the goal.

Real-World Applications

Domain	Agent	Environment	Reward
Games	AI player	board / screen	win / lose / score
Robotics	controller	physical world	task done, no damage
Recommender systems	ranking policy	users	clicks, watch time
Data-centre cooling	controller	temperatures, load	energy saved
LLM fine-tuning (RLHF)	language model	human raters	preference score

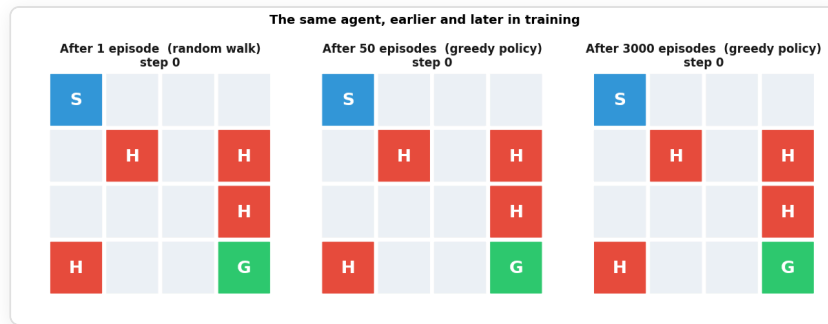
Milestones: Atari from pixels (2013) · AlphaGo (2016) · AlphaStar (2019) · RLHF for ChatGPT (2022)

Now: Live Demo (~8 min)

→ [02-examples/ch10_rl_intro_examples.ipynb](#)

1. Build the GridWorld (16 states, 4 actions, rewards)
2. Random agent — success rate $\approx 1-2\%$
3. Hand-craft a good policy — 100 %, return 0.95
4. Sanity-check the policy for *every* state

Teaser for Ch11: the same agent after 1 / 50 / 3000 training episodes →



Key Takeaways

- RL = learning from **rewards** by trial and error — the agent creates its own data
- Loop: state $\rightarrow$ action $\rightarrow$ reward + next state; **episode, return, policy**
- MDP (S, A, P, R, γ): P = "where do I land?", γ = "how much is the future worth?"
- **Explore vs exploit** — ϵ -greedy with decay
- $Q(s, a)$ = long-term value of an action; know $Q \rightarrow$ policy is `argmax`
- Reward design decides what you get (Bonus exercise: reward shaping)

Next: Chapter 11

Q-Learning

"Now let's teach the agent to fill in the table itself."